

P2 Heurística y Optimización

SAT y CSP



Fernando Mendizábal = 100495773

Alejandro Ladrón de Guevara = 100499541

Índice

1	Introducción	2
2	Descripción de Modelos	2
2.1	Parte 1 - Satisfacción de restricciones con Binairo	2
2.2	Parte 2 - Algoritmos de búsqueda	5
3	Análisis de resultados	10
3.1	Parte 1 - Satisfacción de restricciones con Binairo	10
3.1.1	Conclusión del análisis	12
3.2	Parte 2 - Algoritmos de búsqueda	12
3.2.1	Conclusión del análisis	14

1 Introducción

En este documento presentaremos el trabajo realizado en la modelización y resolución de la tarea de satisfacción de restricciones y del problema de búsqueda heurística planteados. Para ello emplearemos como herramienta la biblioteca de Python llamada "Python Constraint".

2 Descripción de Modelos

2.1 Parte 1 - Satisfacción de restricciones con Binairo

La primera parte de la práctica consiste en la resolución de una tarea de satisfacción de restricciones. En esta, se nos presenta el juego de mesa "Binairo". En él se tiene un tablero de tamaño $n \times n$ casillas, en el que se deben colocar discos blancos o negros en cada casilla de forma que:

- Queden todas las casillas cubiertas.
- El número de discos negros y blancos sea idéntico en cada fila y columna.
- No se puedan disponer más de 2 discos de un color en cada fila o columna.

Para cumplir los siguientes criterios, hemos formalizado el problema de la siguiente forma. Cabe destacar que el problema solo es factible si "n" es un número par. En las variables, "i" representa la fila y "j" representa la columna:

$$R = (V, D, C)$$

$$V = \{x_{i,j}\}_{i,j=1}^n$$

$$D = [0, 1]$$

$$C = \{R_i\}_{i=1}^4$$

**Hemos codificado 0 y 1 para poderlos emplear en el algoritmo, tal que 0 = 0 y 1 = 1.*

Restricción 1: No se pueden disponer más de dos discos de un color de forma consecutiva en una fila.

$$R_1 = \{(x_{i,j}, x_{i,j+1}, x_{i,j+2}) \mid x_{i,j}, x_{i,j+1}, x_{i,j+2} \in D; \\ (x_{i,j} \neq x_{i,j+1} = x_{i,j+2}) \vee (x_{i,j} = x_{i,j+1} \neq x_{i,j+2}) \vee (x_{i,j} \neq x_{i,j+1} \neq x_{i,j+2})\}$$

$$\forall i \in \{1, n\}, \forall j \in \{1, n-2\}$$

Restricción 2: No se pueden disponer más de dos discos de un color de forma consecutiva en una columna.

$$R_2 = \{(x_{i,j}, x_{i+1,j}, x_{i+2,j}) \mid x_{i,j}, x_{i+1,j}, x_{i+2,j} \in D; \\ (x_{i,j} \neq x_{i+1,j} = x_{i+2,j}) \vee (x_{i,j} = x_{i+1,j} \neq x_{i+2,j}) \vee (x_{i,j} \neq x_{i+1,j} \neq x_{i+2,j})\}$$

$$\forall i \in \{1, n-2\}, \forall j \in \{1, n\}$$

Restricción 3: El número de fichas negras y blancas debe ser idéntico en cada fila. Si se cumple que el número de fichas negras es igual a la mitad de "n", por definición se cumple que el número de fichas blancas es igual a la mitad de "n" y por ello tanto fichas blancas como negras están en cada fila en idéntica proporción.

$$R_3 = \{(x_{i,1}, x_{i,2}, \dots, x_{i,n}) \mid x_{i,1}, x_{i,2}, \dots, x_{i,n} \in D; \sum_{j=1}^n x_{i,j} = \frac{n}{2}\}$$

$$\forall i \in \{1, n\}, \frac{n}{2} \in \mathbb{N}$$

Restricción 4: El número de fichas negras y blancas debe ser idéntico en cada columna. Al igual que en la restricción 3, si se cumple que el número de fichas negras es igual a la mitad de "n", por definición se cumple que tanto fichas blancas como negras tienen la misma proporción en cada columna.

$$R_4 = \{(x_{1,j}, x_{2,j}, \dots, x_{n,j}) \mid x_{1,j}, x_{2,j}, \dots, x_{n,j} \in D; \sum_{i=1}^n x_{i,j} = \frac{n}{2}\}$$

$$\forall j \in \{1, n\}, \frac{n}{2} \in \mathbb{N}$$

A continuación, explicaremos la forma en la que hemos presentado y resuelto el problema utilizando las de Python:

Para implementar y resolver el problema de optimización con **Python**, se han utilizado 2 archivos, cada uno con una función específica dentro del flujo de trabajo y las siguientes decisiones de diseño:

Archivo de ejecución de la Parte 1 (parte-1.py)

Lo primero que hace el archivo es comprobar que se ha ejecutado e introducido los 3 argumentos correctamente, saltando un error si no es así. Si se ejecuta adecuadamente el comando, se lee el fichero de entrada (`.in`) de la carpeta de pruebas (si se usa una prueba de ahí) y se extrae de él el contenido del tablero problema. Este tablero se transforma en una matriz de tamaño $n \times n$, donde cada posición representa una casilla que puede estar vacía o contener un disco blanco o negro (O ó X).

A continuación, se imprime por pantalla la instancia a resolver en un formato de tablero delimitado con bordes, para que sea fácil de visualizar. Seguidamente, se realiza una comprobación del tamaño del tablero; si es impar, el script interrumpe la ejecución, ya que no sería posible cumplir la restricción de igualdad de discos blancos y negros por fila y columna, quedando como caso infactible.

Posteriormente, se define el problema de satisfacción de restricciones utilizando la librería `python-constraint`. Cada casilla del tablero se representa como una variable con su dominio correspondiente:

- Las casillas vacías (.) tienen dominio $[0, 1]$, donde 0 representa un disco blanco y 1 un disco negro.
- Las casillas ya ocupadas con un disco blanco O tienen dominio $[0]$.
- Las casillas ocupadas con un disco negro X tienen dominio $[1]$.

Una vez definidas las variables, se añaden las restricciones del juego. La primera y segunda restricciones evitan que haya mas de dos O o X en la misma fila y columna, respectivamente. La tercera y cuarta restricción obligan a que haya el mismo número de O y X en cada fila y columna, de nuevo respectivamente.

Tras definir las variables y restricciones, se procede a resolver el problema utilizando el método `getSolutions()`, el cual resuelve y obtiene todas las soluciones posibles. Se muestra por pantalla el número de soluciones encontradas.

Finalmente, el script escribe en el fichero de salida (`.out`): primero el tablero original, y a continuación una de las soluciones posibles elegida aleatoriamente. El formato de salida mantiene la representación visual del tablero con bordes y casillas, permitiendo diferenciar fácilmente la instancia original de la solución propuesta.

Archivo de entrada (.in)

El archivo de entrada sigue la misma estructura hecha en el enunciado, está compuesto por el tablero a resolver.

Dicho tablero se representa mediante un fichero de texto formado por n líneas, donde cada línea contiene exactamente n caracteres, siendo $n \times n$ el tamaño del tablero. Cada carácter corresponde a una casilla del tablero y define su estado inicial.

Los únicos símbolos permitidos en el fichero son los siguientes:

- ., que indica que el contenido de la casilla es desconocido.
- 0, que indica que la casilla contiene un disco blanco.
- X, que indica que la casilla contiene un disco negro.

2.2 Parte 2 - Algoritmos de búsqueda

La segunda parte de la práctica se centra en la resolución de un problema de búsqueda de caminos en grafos. En este caso, se trata de encontrar la ruta más corta entre dos vértices de un mapa representado por un grafo dirigido, donde los nodos representan ubicaciones sobre la superficie de la Tierra y los arcos representan distancias entre ellos en metros.

Cada instancia del problema se define mediante dos ficheros:

- Un fichero `.gr` que describe los arcos del grafo, indicando los vértices de inicio y fin de cada arco y su coste asociado.
- Un fichero `.co` que proporciona las coordenadas geográficas de cada vértice, permitiendo calcular heurísticas basadas en la distancia euclídea aproximada.

El objetivo consiste en implementar un algoritmo eficiente de búsqueda, capaz de determinar la ruta óptima desde un vértice inicial hasta un vértice final. Para cumplir los siguientes criterios, hemos formalizado el problema de la siguiente forma:

Para la resolución del problema no informada o de fuerza bruta hemos empleado el algoritmo de Dijkstra, que equivale al algoritmo de A^* sin el uso de heurísticas. En cuanto a la resolución informada hemos usado el algoritmo A^* empleando una heurística basada en la distancia euclídea. En la modelización del problema se tiene en cuenta que se está trabajando con un grafo de n nodos o vértices, y cada uno de ellos representa un estado:

- Conjunto de Estados S : Pares de enteros del tipo (x, y) .

* $\{ x = \text{latitud} \}$ y $\{ y = \text{longitud} \}$ de un nodo en el mapa.

- Estado Inicial: $inicial \in S$

- Estado Final: $fin \in S$

Estos estados son idénticos para cualquiera de los dos algoritmos implementados. En el caso del de búsqueda informada se emplea la siguiente heurística. Como se ha mencionado anteriormente, la heurística implementada emplea como medida la distancia euclídea de un nodo a sus vecinos:

$$h(n) = distanciaEuclidea(n, fin) = \sqrt{(x_{fin} - x_n)^2 + (y_{fin} - y_n)^2}$$

Esta heurística es correcta para el problema planteado debido al caracter del mismo, al tratarse de un grafo que representa coordenadas en un mapa. Por ello, en este dominio esta heurística es admisible, porque el resultado obtenido de forma teórico con relajación de restricciones no supera el coste real:

$$h(n) \leq h^*(n)$$

Para cada iteración del algoritmo A*, se determina el coste total del estado inicial hasta el estado final, con valor en $f(n)$. Para ello se le suma a $g(n)$, coste desde el nodo inicial al nodo actual "n", el resultado de la heurística para dicho nodo, que calcula de forma relajada la distancia que faltaría por recorrer hasta el estado final:

$$f(n) = g(n) + h(n)$$

A continuación, explicaremos la forma en la que hemos presentado y resuelto el problema utilizando las de Python:

Para implementar y resolver el problema de optimización con **Python**, se han utilizado 8 archivos, cada uno con una función específica dentro del flujo de trabajo y las siguientes decisiones de diseño:

Archivo de ejecución de la Parte 2 (parte-2.py)

El archivo `parte-2.py` actúa como script principal de la Parte 2 y se encarga de coordinar la lectura de los datos de entrada, la ejecución del algoritmo de búsqueda y la generación de la salida.

Lo primero que realiza el script es comprobar que se han introducido correctamente los cuatro argumentos requeridos en la línea de comandos: el vértice inicial, el vértice final, el nombre del mapa y el fichero de salida. En caso contrario, se muestra un mensaje de uso y se interrumpe la ejecución.

A continuación, se ajustan las rutas de los ficheros de mapa y de salida, buscando en el directorio `pruebas-2` si no se especifica una ruta absoluta.

Seguidamente, se crea un objeto de la clase **Grafo**, encargada de leer y almacenar en memoria la información del grafo. Se leen los ficheros `.gr`, que contienen los arcos y

sus costes, y `.co`, que contienen las coordenadas de los vértices. Una vez procesados, se muestra por pantalla el número total de vértices y el número de arcos del grafo.

Tras la carga del grafo, se instancia la clase `Algoritmo`, que implementa el algoritmo de búsqueda elegido. En este caso, se emplea el algoritmo A^* , iniciando la búsqueda del camino más corto entre el vértice inicial y el final. Durante este proceso se mide el tiempo de ejecución y se contabiliza el número de nodos expandidos.

Una vez finalizada la búsqueda, el script muestra por pantalla el coste de la solución óptima encontrada, el tiempo total de ejecución y el número de expansiones realizadas. Finalmente, se escribe la solución en el fichero de salida especificado (`.out`), indicando de forma secuencial los vértices del camino junto con el coste de cada arco. En caso de que no exista un camino válido entre los vértices indicados, es decir, el escenario es infactible, se escribe en el fichero el mensaje `Caso infactible`.

Archivo `abierta.py`: lista abierta de nodos a explorar

El archivo `abierta.py` define la clase `Abierta`, que representa la lista abierta utilizada en los algoritmos de búsqueda A^* o Dijkstra. Internamente, se utiliza un `heap` (cola de prioridad) para almacenar los nodos pendientes de explorar, de forma que siempre se pueda extraer de manera eficiente el nodo con menor coste total.

La clase proporciona los siguientes métodos principales:

- `push(coste_total, nodo, path)`: añade un nodo al `heap` manteniendo el orden por coste total.
- `pop()`: extrae y devuelve el nodo con menor coste total.
- `vacía()`: comprueba si la lista abierta está vacía.

Archivo `cerrada.py`: lista cerrada de nodos visitados

El archivo `cerrada.py` define la clase `Cerrada`, que representa la lista cerrada utilizada en los algoritmos de búsqueda A^* o Dijkstra. Esta lista guarda los nodos que ya han sido visitados para evitar procesarlos de nuevo.

La clase proporciona los siguientes métodos principales:

- `add(nodo)`: añade un nodo al conjunto de nodos visitados.
- `contiene(nodo)`: comprueba si un nodo ya ha sido visitado, devolviendo `True` en caso de que sí o `False` en caso contrario.

Archivo `grafo.py`: representación del grafo

El archivo `grafo.py` define la clase `Grafo`, que se encarga de representar un grafo dirigido con arcos y coordenadas asociadas a cada nodo.

La clase contiene dos diccionarios principales:

- **arcos**: guarda los arcos de cada nodo, donde la clave es el nodo origen y el valor es una lista de tuplas (`nodo destino`, `coste`).
- **coordenadas**: almacena las coordenadas de cada nodo, la clave, como tuplas (`latitud`, `longitud`), el valor.

Los métodos principales son:

- `leer_gr(archivo_gr)`: lee un fichero `.gr` y construye los arcos del grafo.
- `leer_co(archivo_co)`: lee un fichero `.co` y guarda las coordenadas de los nodos.
- `vecinos(nodo)`: devuelve la lista de vecinos y costes de un nodo dado, o una lista vacía si el nodo no tiene arcos.

Archivo `algoritmo.py`: algoritmo de búsqueda

El archivo `algoritmo.py` define la clase `Algoritmo`, que implementa los algoritmos de búsqueda de caminos en un grafo: A^* (con heurística euclídea) y Dijkstra (equivalente a A^* sin heurística). La clase mantiene un contador de nodos expandidos y almacena un objeto `Grafo` con los arcos y coordenadas de los nodos.

Se inicializa recibiendo un objeto `Grafo` y lo almacena en el atributo `grafo`. Además, inicializa el contador `nodos_expandidos` a cero, que se irá actualizando durante la ejecución de los algoritmos para llevar un registro del número de nodos visitados.

Los métodos principales son los siguientes:

- `heuristica(u, v)`: calcula la distancia aproximada entre dos nodos usando sus coordenadas geográficas, latitud y longitud convertidas a metros. Esta distancia euclídea se utiliza como heurística para A^* , permitiendo estimar el coste restante hasta el destino.
- `buscar_A_estrella(inicio, fin)`: implementa el algoritmo A^* , manteniendo una lista abierta de nodos por explorar y una lista cerrada de nodos ya visitados. En cada iteración se extrae el nodo con menor coste estimado $f = g + h$, donde g es el coste acumulado desde el nodo inicial y h la heurística hasta el nodo final. Para cada vecino no visitado o con un camino más barato, se actualiza el coste y se añade

a la lista abierta junto con la ruta parcial, que mantiene el camino desde el nodo inicial hasta cada vecino. Devuelve la ruta óptima, el coste total y el número de nodos expandidos.

- `buscar_Dijkstra(inicio, fin)`: implementa Dijkstra delegando en `buscar_A_estrella_sin_heuristica`, que es una variante de A^* donde el coste estimado se basa únicamente en g , es decir, sin utilizar la heurística. Internamente funciona igual que A^* , pero garantiza que la exploración se realiza únicamente por coste acumulado, permitiendo encontrar siempre el camino más corto.
- `buscar_A_estrella_sin_heuristica(inicio, fin)`: idéntico a `buscar_A_estrella`, pero el coste total f se calcula solo como g . Se utiliza para implementar Dijkstra y permite comparar el rendimiento con A^* , mostrando la diferencia en número de nodos expandidos entre usar o no la heurística.

Archivo `crear-aislado.py`: método de aislamiento de vertices

El archivo `crear-aislado.py` permite generar una nueva versión de un grafo en la que un vértice específico queda completamente aislado, eliminando todos sus arcos entrantes y salientes. Para ello, se define la función `crear_vertice_aislado(nombre_mapa, vertice_a_aislar, output_prefix)`:

El fichero primero construye las rutas de los archivos originales `.gr` y `.co` dentro de la carpeta `pruebas-2` y comprueba que existen. A continuación, crea un objeto `Grafo` y carga los datos de arcos y coordenadas mediante los métodos `leer_gr` y `leer_co` (de la clase `Grafo`). Posteriormente, elimina todos los arcos salientes del vértice a aislar y todos los arcos entrantes que apunten a él. Finalmente, define los nombres de los archivos de salida con el prefijo indicado en la llamada a la función y guarda el archivo `.gr` modificado con los arcos actualizados y el archivo `.co` con las coordenadas originales, manteniendo el formato DIMACS.

El script incluye además un bloque principal que permite ejecutarlo desde la línea de comandos, comprobando que se reciben exactamente tres argumentos: nombre del mapa, vértice a aislar y prefijo de salida, antes de llamar a la función principal.

Archivos de entrada de arcos y coordenadas (`.gr`) y (`.co`)

Los archivos de entrada para los scripts de manipulación y búsqueda en grafos son de dos tipos: `.gr` y `.co`. El archivo `.gr` contiene la información de los arcos del grafo, representando cada línea un arco con el formato `a nodo_origen nodo_destino coste`, donde `nodo_origen` y `nodo_destino` son enteros que identifican los vértices y `coste` indica el peso o distancia del arco entre ambos nodos. Este archivo puede contener múltiples arcos para cada nodo, reflejando todas las conexiones salientes del grafo.

El archivo `.co` almacena las coordenadas geográficas de cada vértice en formato DIMACS, con líneas con el formato `v nodo lat lon`, donde `nodo` es el entero que identifica al vértice

y, `lat` y `lon` las coordenadas de latitud y longitud del vértice en el mapa, respectivamente. Estas coordenadas se utilizan para calcular distancias aproximadas y heurísticas en los algoritmos de búsqueda, como A^* , y permiten representar espacialmente los nodos del grafo.

3 Análisis de resultados

Ahora haremos un análisis de los resultados obtenidos en cada parte. Probaremos distintos casos de uso para nuestros problemas y veremos como actúan los modelos de acuerdo a ellos.

3.1 Parte 1 - Satisfacción de restricciones con Binairo

A continuación se presentan los casos de uso más relevantes para analizar el comportamiento del modelo:

Caso tablero 4x4 resoluble: El tablero de entrada tiene tamaño 4×4 , cumpliendo la condición de paridad necesaria para el problema. La instancia contiene casillas parcialmente definidas, sin violar ninguna restricción inicial del juego, por lo que es factible y el modelo debería poder resolverlo.

Con el archivo de entrada `prueba-1.in` vemos que el modelo produce este resultado:

Al resolver el tablero se encuentran 11 soluciones distintas al problema, por lo cual no tiene solución única. Todas cumpliendo con las restricciones del problema, como la limitación de `X` y `O` seguidos en columnas y filas, o la acotación de `X` y `O` a haber el mismo número de ambos. También, el tiempo de ejecución es mínimo ya que es una instancia muy pequeña y con no muchas soluciones.

Caso tablero 6x6 resoluble: El tablero de entrada tiene tamaño 6×6 , cumpliendo la condición de paridad necesaria para el problema. La instancia contiene casillas parcialmente definidas, sin violar ninguna restricción inicial del juego, por lo que es factible y el modelo debería poder resolverlo.

Con el archivo de entrada `prueba2.in` vemos que el modelo produce este resultado:

Al resolver el tablero se encuentran 9 soluciones distintas al problema, por lo cual no tiene solución única. Todas cumpliendo con las restricciones del problema, como la limitación de `X` y `O` seguidos en columnas y filas, o la acotación de `X` y `O` a haber el mismo número de ambos. También, el tiempo de ejecución sigue siendo bajo ya que la instancia es pequeña y con un número reducido de soluciones.

Caso tablero 6x6 vacío: El tablero de entrada tiene tamaño 6×6 y todas sus casillas están inicialmente vacías, cumpliendo la condición de paridad necesaria para el problema. Al no haber ninguna casilla predefinida, la instancia es totalmente abierta y el modelo debería poder explorar todas las posibles configuraciones factibles.

Con el archivo de entrada `prueba3.in` vemos que el modelo produce este resultado:
Al resolver el tablero se encuentran 11,222 soluciones distintas al problema, por lo cual no tiene solución única. Todas cumplen con las restricciones del juego, como la limitación de X y O seguidos en columnas y filas, y la necesidad de mantener el mismo número de X y O. El tiempo de ejecución es mayor, aunque sigue siendo de unos pocos segundos, ya que el tablero inicial estaba vacío y el modelo puede explorar todas las combinaciones posibles.

Caso tablero 4x4 no resoluble: El tablero de entrada tiene tamaño 4×4 y contiene filas completamente definidas para X y O en la mitad superior, es decir, las dos primeras filas están formadas por 4 X y 4 O respectivamente, mientras que las filas inferiores están vacías. Esta configuración rompe la posibilidad de cumplir con las restricciones de paridad en el juego, por lo que la instancia no es resoluble.

Con el archivo de entrada `prueba4.in` vemos que el modelo produce este resultado:
Al resolver el tablero no se encuentra ninguna solución factible. Esto confirma que el modelo identifica correctamente las instancias que no cumplen las restricciones iniciales del juego. El tiempo de ejecución es mínimo, ya que el algoritmo detecta rápidamente la imposibilidad de generar una solución.

Caso tablero 4x4 con solución única: El tablero de entrada tiene tamaño 4×4 y contiene algunas casillas predefinidas alternadas, de manera que la instancia cumple la condición de paridad, es resoluble y permite únicamente una configuración final que cumpla todas las restricciones del juego.

Con el archivo de entrada `prueba5.in` vemos que el modelo produce este resultado:
Al resolver el tablero se encuentra 1 solución única al problema, cumpliendo todas las restricciones, como la limitación de X y O seguidos en columnas y filas y la igualdad de número de X y O. El tiempo de ejecución es mínimo, ya que la instancia es pequeña y la solución única permite que el algoritmo complete el cálculo de forma muy rápida.

Caso tablero 10x10 parcialmente definido: El tablero de entrada tiene tamaño 10×10 con casillas predefinidas alternadas que cumplen la condición de paridad del juego. La instancia es algo más compleja, debido a su tamaño, pero factible, ya que las posiciones iniciales no violan ninguna restricción del problema.

Con el archivo de entrada `prueba6.in` vemos que el modelo produce este resultado:
Al resolver el tablero se encuentran 216 soluciones distintas al problema, por lo cual no tiene solución única. Todas las soluciones cumplen con las restricciones del juego, incluyendo la limitación de X y O seguidos en columnas y filas y el equilibrio de número de X y O. El tiempo de ejecución es mayor que en los tableros más pequeños, pero sigue siendo razonable, reflejando la mayor complejidad de esta instancia debido al tamaño del tablero y al número de combinaciones posibles.

Caso tablero 5x5 vacío (impar): El tablero de entrada tiene tamaño 5×5 , cumpliendo la condición de estar completamente vacío. Sin embargo, al ser de tamaño impar, no cumple la condición de paridad necesaria para el problema, por lo que la instancia no es resoluble y el modelo no genera ninguna solución.

Con el archivo de entrada `prueba7.in`, el modelo detecta inmediatamente que el tablero tiene tamaño impar y por ello no se ejecuta el algoritmo de resolución. No se produce ningún archivo de salida `.out` tampoco.

3.1.1 Conclusión del análisis

Del análisis de los casos se pueden extraer las siguientes conclusiones:

- **Restricciones y factibilidad:** Las restricciones de paridad, X/O seguidos y equilibrio de X y O determinan si un tablero es resoluble. Tableros que violan estas restricciones, como el 4x4 no resoluble o el 5x5 vacío (impar), no generan soluciones, mientras que tableros parcialmente definidos guían correctamente la generación de soluciones válidas.
- **Influencia de las casillas predefinidas y tamaño del tablero:** El estado de las casillas iniciales delimita el espacio de soluciones y la exploración necesaria. Tableros con pocas casillas predefinidas, como el 6x6 vacío, generan muchas combinaciones (11 222 soluciones) y tiempos mayores. Tableros más grandes parcialmente definidos (10x10) generan cientos de soluciones con tiempos aún razonables. Tableros pequeños parcialmente definidos (4x4 o 6x6) requieren tiempos mínimos, casi instantáneos.
- **Variables y restricciones:** Cada casilla del tablero se modela como una variable binaria (X o O). Las restricciones principales son:
 1. Paridad por filas: igual número de X y O en cada fila.
 2. Paridad por columnas: igual número de X y O en cada columna.
 3. Secuencia en fila: máximo dos X o O consecutivos en cada fila.
 4. Secuencia en columna: máximo dos X o O consecutivos en cada columna.
- **Progresión del tiempo de ejecución y complejidad:** El tiempo crece con el tamaño del tablero y el número de casillas vacías debido al aumento combinatorio. Por ejemplo, pasar de un tablero 4x4 parcialmente definido a un 6x6 vacío incrementa las soluciones de 11 a 11 222 y el tiempo de ejecución de milisegundos a varios segundos. La instancia más grande resuelta fue el 10x10 parcialmente definido (216 soluciones).

En resumen, el modelo cumple su objetivo de generar configuraciones de X y O que respetan todas las restricciones, detectando correctamente instancias no resolubles y adaptándose eficientemente a tableros de distintos tamaños y niveles de definición inicial. La complejidad y el tiempo de ejecución aumentan principalmente con el tamaño del tablero y el número de casillas vacías, reflejando el crecimiento combinatorio del problema.

3.2 Parte 2 - Algoritmos de búsqueda

A continuación se presentan los casos de uso más relevantes para analizar el comportamiento del modelo:

Caso base – recorrido corto entre dos nodos cercanos: Este caso plantea el camino más corto entre dos vértices cercanos del mapa de la Bay Area. La instancia es factible, ya que ambos nodos pertenecen a la misma zona del grafo y existe conectividad.

Con los archivos `USA-road-d.BAY.co` y `USA-road-d.BAY.gr`, buscando el camino entre los nodos 1 y 5, el algoritmo procesa el grafo completo (321 270 vértices y 800 172 arcos) y encuentra una solución óptima de 18 632 metros. El número de nodos expandidos es muy reducido (12), con un tiempo de ejecución prácticamente inmediato.

El fichero de salida `.out` confirma el camino óptimo, compuesto por 5 saltos entre vértices, cuya suma de costes da lugar al total de la solución.

Caso extremo – recorrido máximo entre dos nodos distantes: Este caso plantea el camino más corto entre dos vértices situados en extremos opuestos del mapa de la Bay Area. La instancia es factible, ya que ambos nodos pertenecen al grafo y existe conectividad, pero la gran distancia aumenta la complejidad de la búsqueda.

Con los archivos `USA-road-d.BAY.co` y `USA-road-d.BAY.gr`, buscando el camino entre los nodos 1 y 321 270, el algoritmo procesa el grafo completo (321 270 vértices y 800 172 arcos) y encuentra una solución óptima de 1 853 714 metros. El número de nodos expandidos es muy elevado (307 709), con un tiempo de ejecución de 1.90 segundos.

El fichero de salida `.out` confirma el camino óptimo, compuesto por 307 saltos entre vértices, cuya suma de costes da lugar al total de la solución.

Caso infactible – vértice inaccesible: Este caso plantea el camino más corto entre dos vértices, siendo uno completamente inaccesible. La instancia se genera con el script `crear-aislado.py`, que aísla un vértice del resto del grafo, garantizando que no existe camino factible.

El algoritmo procesa el grafo completo (321 270 vértices y 800 160 arcos) y detecta correctamente la ausencia de solución, devolviendo un coste infinito (`inf`). La búsqueda explora casi todo el grafo, con 321 268 nodos expandidos, y el tiempo de ejecución es de 1.92 segundos.

El fichero de salida `.out` refleja explícitamente el caso infactible, mostrando “Caso infactible”, ya que no hay camino a reportar.

Caso puentes – recorrido por puentes entre dos nodos separados por agua: Este caso plantea el camino más corto entre dos vértices situados en masas de tierra distintas separadas por agua. Dada la dificultad de encontrar los vertices exactos manualmente, se identificaron los vértices del puente de San Mateo (Foster City–Hayward) utilizando Google Maps, ya que usa el mismo sistema de coordenadas que nuestros mapas, para comprobar que el algoritmo lo atraviesa.

Con los archivos `USA-road-d.BAY.co` y `USA-road-d.BAY.gr`, buscando el camino entre los nodos 15803 y 37741, el algoritmo procesa el grafo completo (321 270 vértices y 800 172 arcos) y encuentra una solución óptima de 85 673 metros con 6 937 expansiones. La única forma de recorrer esta distancia con tan pocas expansiones es mediante el puente.

El fichero de salida `.out` confirma el camino óptimo, compuesto por 73 saltos entre vértices, cuya suma de costes da lugar al total de la solución.

3.2.1 Conclusión del análisis

Del análisis de los casos de la búsqueda de camino mínimo se pueden extraer las siguientes conclusiones:

- **Factibilidad y corrección de la solución:** El algoritmo A* resuelve correctamente instancias factibles e infactibles. En los casos base, extremo y puentes se obtiene siempre un camino óptimo cuyo coste coincide con la distancia mínima. En el caso infactible, detecta la ausencia de solución, devolviendo un coste infinito (`inf`).
- **Influencia de la distancia entre nodos:** La distancia entre el nodo origen y el destino determina la dificultad de la búsqueda. Para nodos cercanos (caso base) se requieren pocas expansiones (12) con tiempo prácticamente inmediato; para nodos lejanos (caso extremo) se superan las 300 000 expansiones y el tiempo aumenta a aproximadamente 1.9 segundos. En instancias infactibles, se explora casi todo el grafo (321 268 expansiones) con tiempo similar al caso extremo.
- **Uso de heurística y reducción del espacio de búsqueda:** La heurística euclídea de A* reduce significativamente el número de nodos expandidos frente a una búsqueda no informada. En el caso de los puentes, guía la búsqueda hacia el atajo correcto, reduciendo las expansiones a 6 937, mostrando su eficacia para priorizar regiones prometedoras del grafo.
- **Estructuras de datos, complejidad y tamaño máximo resuelto:** Las estructuras (listas abierta y cerrada, costes y predecesores) crecen proporcionalmente al número de nodos expandidos, llegando en el peor caso a contener prácticamente todos los vértices del grafo (321 270 vértices y más de 800 000 arcos). El tiempo de ejecución depende de la distancia entre nodos y de la heurística, resolviendo incluso las instancias más grandes en menos de 2 segundos.

En resumen, A* con heurística euclídea es adecuado para resolver el problema de búsqueda de camino mínimo, encontrando soluciones óptimas cuando existen y detectando correctamente los casos infactibles. La complejidad y el tiempo de ejecución están dominados principalmente por el número de nodos expandidos, dependiente de la distancia entre nodos y de la capacidad de la heurística para guiar la búsqueda.